

Architecture & Trade-offs

In-Memory Account Ledger Core

Architecture decisions, trade-offs, and production considerations arising from this ledger implementation. This document does not restate the event stream or Part 1 rule text.

1. Append-only at scale

What breaks first at ~100x volume?

Balance-as-
backdated
CPU/latenc
larger mult
ceiling.

Where unbounded state accumulates

The appen
We recom

Cheapest structural change that defers the problem

Keep appe
structure) u
scan. Optic

2. Value-dated entries in production (UAE-licensed bank)

Operational / regulatory surface this design creates

A booking
immediatel
earlier valu

- Customer statement restatements and complaints

- Profit / int

One control before go-live

Maker-che
a closed
auto-gener

3. Authorization lifecycle - endings other than matching settlement

In code today an authorization is active until settled (or never approved). Production must define every other ending:

Expiry / TTL

Scenario: C

Mandate: Auto-release hold; append AUTH_EXPIRED; available increases; no ledger debit

Merchant void / reverse

Scenario: M

Mandate: AUTH_RELEASE; release hold; no debit

Partial capture then close

Scenario: C

Mandate: Debit capture amount; release entire remaining hold

Over-capture (settle > hold)

Scenario: M

Mandate: Not implemented here. Mandate: reject, or require incremental auth first; if scheme allows over-capture, check available for the excess and error when insufficient

Incremental replace

Scenario: H

Mandate: Replace hold amount only if available allows; else reject

Force clear / ops

Scenario: F

Mandate: Privileged AUTH_FORCE_RELEASE with audit; never silent delete

Account closure

Scenario: C

Mandate: Block closure until holds clear, or force-release under policy then close

Part 1 leaves open-ended holds unimplemented (and Auth-B is rejected by available rules after a backdated debit). That is a scope cut, not a claim that open holds are safe in production.

4. What you cut and why

No persistence / single process

Why: Asse

Risk deferred: Restart loses the ledger; no durability / HA

No concurrency control

Why: Singl

Risk deferred: Parallel auth races on available balance

No FX / cross-currency

Why: Acco

Risk deferred: Cross-currency auth/settle undefined

No fee cascade on reversal

Why: Appe

Risk deferred: Customer may keep OD fees after correcting debit

No auth expiry / void in code

Why: Strea

Risk deferred: Stuck holds starve available

No settle > hold guard / excess-available check

Why: Strea

Risk deferred: Over-capture could debit unreserved funds without an error

No double-entry GL

Why: Singl

Risk deferred: Cannot prove books balance to bank GL

No idempotency keys

Why: Deter

Risk deferred: Duplicate posts on retry

Full-scan balances

Why: Six-d

Risk deferred: Latency collapse at volume (section 1)

Interest capitalizes once at window end

Why: Six-d

Risk deferred: Mismatch vs continuous / period production schedules

Every claim above maps to the accompanying repository: balance scans in ledger.ts / policies/overdraft.ts, holds in authorizations.ts, cuts visible as absent modules (no database, no TTL worker, no general ledger).